Why was it important to separate these two responsibilities into separate classes? Because each responsibility is an axis of change. When the requirements change, that change will be manifest through a change in responsibility amongst the classes. If a class assumes more than one responsibility, then there will be more than one reason for it to change.

If a class has more than one responsibility, then the responsibilities become coupled. Changes to one responsibility may impair or inhibit the ability of the class to meet the others. This kind of coupling leads to fragile designs that break in unexpected ways when changed.

For example, consider the design in Figure 8-1. The `Rectangle` class has two methods shown. One draws the rectangle on the screen, the other computes the area of the rectangle.

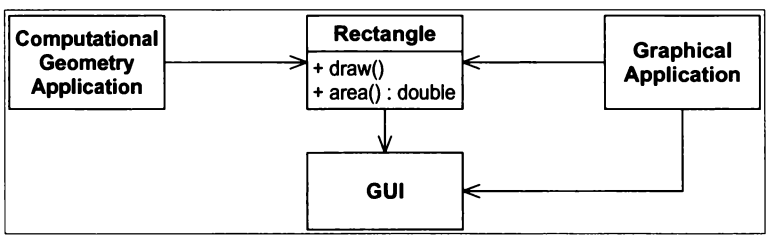


Figure 8-1 More than one responsibility

Two different applications use the `Rectangle` class. One application does computational geometry. It uses `Rectangle` to help it with the mathematics of geometric shapes. It never draws the rectangle on the screen. The other application is graphical in nature. It may also do some computational geometry, but it definitely draws the rectangle on the screen.

This design violates the Single-Responsibility Principle (SRP). The `Rectangle` class has two responsibilities. The first responsibility is to provide a mathematical model of the geometry of a rectangle. The second responsibility is to render the rectangle on a graphical user interface.

This violation of the SRP causes several nasty problems. First, we must include the `GUI` in the computational geometry application. If this were a C++ application, the GUI would have to be linked in, consuming link time, compile time, and memory footprint. In a Java application, the `.class` files for the `GUI` have to be deployed to the target platform.

Second, if a change to the `GraphicalApplication` causes the `Rectangle` to change for some reason, that change may force us to rebuild, retest, and redeploy the `ComputationalGeometryApplication`. If we forget to do this, that application may break in unpredictable ways.

A better design is to separate the two responsibilities into two completely different classes as shown in Figure 8-2. This design moves the computational portions of `Rectangle` into the `GeometricRectangle` class. Now changes made to the way rectangles are rendered cannot affect the `ComputationalGeometryApplication`.

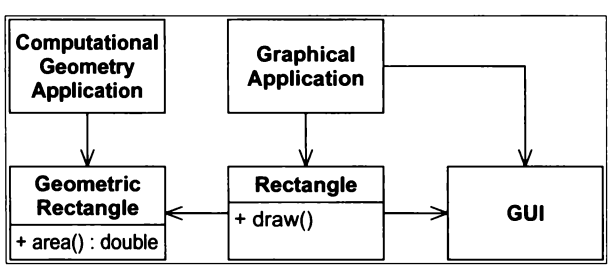


Figure 8-2 Separated Responsibilities

What Is a Responsibility?

In the context of the SRP, we define a responsibility to be "a reason for change." If you can think of more than one motive for changing a class, then that class has more than one responsibility. This is sometimes hard to see. We are accustomed to thinking of responsibility in groups. For example, consider the `Modem` interface in Listing 8-1. Most of us will agree that this interface looks perfectly reasonable. The four functions it declares are certainly functions belonging to a modem.

Listing 8-1

`Modem.java -- SRP Violation`

```
interface Modem
{
  public void dial(String pno);
  public void hangup();
  public void send(char c);
  public char recv();
}
```

However, there are two responsibilities being shown here. The first responsibility is connection management. The second is data communication. The `dial` and `hangup` functions manage the connection of the modem, while the `send` and `recv` functions communicate data.

Should these two responsibilities be separated? That depends on how the application is changing. If the application changes in ways that affect the signature of the connection functions, then the design will smell of Rigidity because the classes that call `send` and `recv` will have to be recompiled and redeployed more often than we like. In that case the two responsibilities should be separated as shown in Figure 8-3. This keeps the client applications from coupling the two responsibilities.

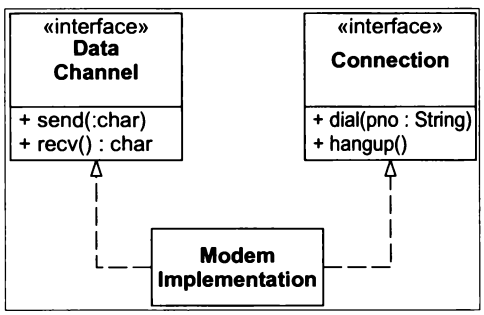


Figure 8-3 Separated Modem Interface

If, on the other hand, the application is not changing in ways that cause the the two responsibilities to change at different times, then there is no need to separate them. Indeed, separating them would smell of Needless Complexity.

There is a corollary here. *An axis of change is an axis of change only if the changes actually occur.* It is not wise to apply the SRP, or any other principle for that matter, if there is no symptom.

Separating Coupled Responsibilities

Notice that in Figure 8-3 I kept both responsibilities coupled in the `ModemImplementation` class. This is not desirable, but it may be necessary. There are often reasons, having to do with the details of the hardware or OS, that force us to couple things that we'd rather not couple. However, by separating their interfaces we have decoupled the concepts as far as the rest of the application is concerned.

We may view the `ModemImplementation` class as a kludge, or a wart; however, notice that all dependencies flow *away* from it. Nobody needs to depend on this class. Nobody except `main` needs to know that it exists. Thus, we've put the ugly bit behind a fence. Its ugliness need not leak out and pollute the rest of the application.

Persistence

Figure 8-4 shows a common violation of the SRP. The `Employee` class contains business rules and persistence control. These two responsibilities should almost never be mixed. Business rules tend to change frequently, and though persistence may not change as frequently, it changes for completely different reasons. Binding business rules to the persistence subsystem is asking for trouble.

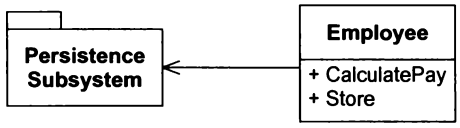


Figure 8-4 Coupled Persistence

Fortunately, as we saw in Chapter 4, the practice of test-driven development will usually force these two responsibilities to be separated long before the design begins to smell. However, in cases where the tests did not force the separation, and the smells of Rigidity and Fragility become strong, the design should be refactored using the FACADE or PROXY patterns to separate the two responsibilities.

Conclusion

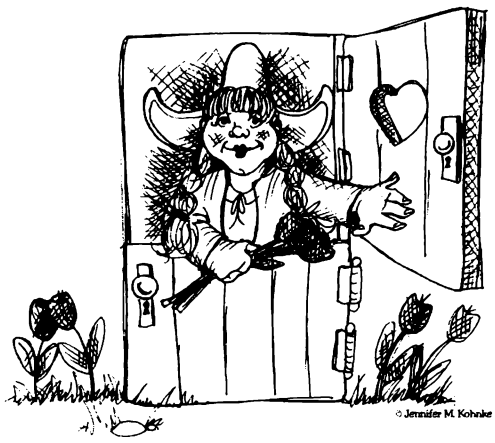
The SRP is one of the simplest of the principles, and one of the hardest to get right. Conjoining responsibilities is something that we do naturally. Finding and separating those responsibilities from one another is much of what software design is really about. Indeed, the rest of the principles we will discuss come back to this issue in one way or another.

Bibliography

1. DeMarco, Tom. *Structured Analysis and System Specification.* Yourdon Press Computing Series. Englewood Cliff, NJ: 1979.
2. Page-Jones, Meilir. *The Practical Guide to Structured Systems Design*, 2d ed. Englewood Cliff, NJ: Yourdon Press Computing Series, 1988.

9

OCP: The Open–Closed Principle



Dutch Door—*(Noun) A door divided in two horizontally so that either part can be left open or closed.*

—The American Heritage® Dictionary of the English Language: Fourth Edition. 2000.

As Ivar Jacobson has said, "All systems change during their life cycles. This must be born in mind when developing systems are expected to last longer than the first version."[1] How can we create designs that are stable in the face of change and that will last longer than the first version? Bertrand Meyer gave us guidance as long ago as 1988 when he coined the now famous Open–Closed Principle.[2]

OCP: The Open–Closed Principle

> *Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.*

When a single change to a program results in a cascade of changes to dependent modules, the design smells of Rigidity. The OCP advises us to refactor the system so that further changes of that kind will not cause more

1. [Jacobson92], p. 21.
2. [Meyer97], p. 57.

modifications. If the OCP is applied well, then further changes of that kind are achieved by adding new code, not by changing old code that already works.

This may seem like motherhood and apple pie—the golden unachievable ideal—but in fact there are some relatively simple and effective strategies for *approaching* that ideal.

Description

Modules that conform to the Open–Closed Principle have two primary attributes. They are

1. "Open for extension."
   This means that the behavior of the module can be extended. As the requirements of the application change, we are able to extend the module with new behaviors that satisfy those changes. In other words, we are able to change what the module does.
2. "Closed for modification."
   Extending the behavior of a module does not result in changes to the source or binary code of the module. The binary executable version of the module, whether in a linkable library, a DLL, or a Java `.jar`, remains untouched.

It would seem that these two attributes are at odds with each other. The normal way to extend the behavior of a module is to make changes to the source code of that module. A module that cannot be changed is normally thought to have a fixed behavior.

How is it possible that the behaviors of a module can be modified without changing its source code? How can we change what a module does, without changing the module?

Abstraction Is the Key

In C++, Java, or any other OOPL,[3] it is possible to create abstractions that are fixed and yet represent an unbounded group of possible behaviors. The abstractions are abstract base classes, and the unbounded group of possible behaviors is represented by all the possible derivative classes.

It is possible for a module to manipulate an abstraction. Such a module can be closed for modification since it depends upon an abstraction that is fixed. Yet the behavior of that module can be extended by creating new derivatives of the abstraction.

Figure 9–1 shows a simple design that does not conform to the OCP. Both the `Client` and `Server` classes are concrete. The `Client` class *uses* the `Server` class. If we wish for a `Client` object to use a different server object, then the `Client` class must be changed to name the new server class.

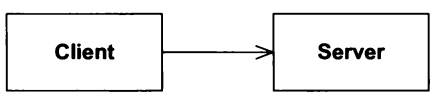


Figure 9-1 Client is not open and closed

Figure 9–2 shows the corresponding design that conforms to the OCP. In this case, the `ClientInterface` class is an abstract class with abstract member functions. The `Client` class uses this abstraction; however, objects of the `Client` class will be using objects of the derivative `Server` class. If we want `Client` objects to use a different server class, then a new derivative of the `ClientInterface` class can be created. The `Client` class can remain unchanged.

The `Client` has some work that it needs to get done, and it can describe that work in terms of the abstract interface presented by `ClientInterface`. Subtypes of `ClientInterface` can implement that interface in any

3. Object-oriented programming language.

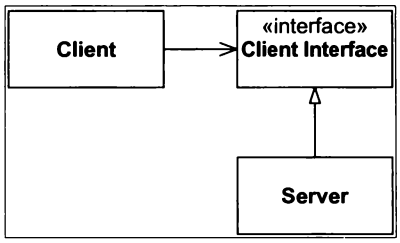


Figure 9-2 STRATEGY pattern: Client is both open and closed

manner they choose. Thus, the behavior specified in `Client` can be extended and modified by creating new subtypes of `ClientInterface`.

You may wonder why I named `ClientInterface` the way I did. Why didn't I call it `AbstractServer` instead? The reason, as we will see later, is that *abstract classes are more closely associated to their clients than to the classes that implement them.*

Figure 9-3 shows an alternative structure. The `Policy` class has a set of concrete public functions that implements a policy of some kind. Similar to the functions of the `Client` in Figure 9-2. As before, those policy functions describe some work that needs to be done in terms of some abstract interfaces. However, in this case, the abstract interfaces are part of the `Policy` class itself. In C++ they would be pure virtual functions, and in Java they would be abstract methods. Those functions are implemented in the subtypes of `Policy`. Thus, the behaviors specified within `Policy` can be extended or modified by creating new derivatives of the `Policy` class.

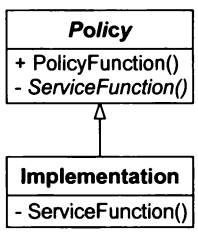


Figure 9-3 Template Method Pattern: Base class is open and closed

These two patterns are the most common ways of satisfying the OCP. They represent a clear separation of generic functionality from the detailed implementation of that functionality.

The `Shape` Application

The following example has been shown in many books on OOD. It is the infamous "Shape" example. It is normally used to show how polymorphism works. However, this time we will use it to elucidate the OCP.

We have an application that must be able to draw circles and squares on a standard GUI. The circles and squares must be drawn in a particular order. A list of the circles and squares will be created in the appropriate order, and the program must walk the list in that order and draw each circle or square.

Violating the OCP

In C, using procedural techniques that do not conform to the OCP, we might solve this problem as shown in Listing 9-1. Here we see a set of data structures that has the same first element, but is different beyond that. The first element of each is a type code that identifies the data structure as either a circle or a square. The function `DrawAllShapes` walks an array of pointers to these data structures, examining the type code and then calling the appropriate function (either `DrawCircle` or `DrawSquare`).

Listing 9-1

Procedural Solution to the Square/Circle Problem

```
--shape.h-------------------------------------
enum ShapeType {circle, square};

struct Shape
{
  ShapeType itsType;
};

--circle.h------------------------------------
struct Circle
{
  ShapeType itsType;
  double itsRadius;
  Point itsCenter;
};

void DrawCircle(struct Circle*);

--square.h------------------------------------
struct Square
{
  ShapeType itsType;
  double itsSide;
  Point itsTopLeft;
};

void DrawSquare(struct Square*);

--drawAllShapes.cc-----------------------------
typedef struct Shape *ShapePointer;

void DrawAllShapes(ShapePointer list[], int n)
{
  int i;
  for (i=0; i<n; i++)
  {
    struct Shape* s = list[i];
    switch (s->itsType)
    {
    case square:
      DrawSquare((struct Square*)s);
    break;

    case circle:
      DrawCircle((struct Circle*)s);
    break;
    }
  }
}
```

The function `DrawAllShapes` does not conform to the OCP because it cannot be closed against new kinds of shapes. If I wanted to extend this function to be able to draw a list of shapes that included triangles, I would

have to modify the function. In fact, I would have to modify the function for any new type of shape that I needed to draw.

Of course this program is only a simple example. In real life, the `switch` statement in the `DrawAllShapes` function would be repeated over and over again in various functions all through the application, each one doing something a little different. There might be functions for dragging shapes, stretching shapes, moving shapes, deleting shapes, etc. Adding a new shape to such an application means hunting for every place that such `switch` statements (or `if/else` chains) exist and adding the new shape to each.

Moreover, it is very unlikely that all the `switch` statements and `if/else` chains would be as nicely structured as the one in `DrawAllShapes`. It is much more likely that the predicates of the `if` statements would be combined with logical operators or that the `case` clauses of the `switch` statements would be combined so as to "simplify" the local decision making. In some pathological situations, there may be functions that do precisely the same things to `Squares` that they do to `Circles`. Such functions would not even have the `switch/case` statements or `if/else` chains. Thus, the problem of finding and understanding all the places where the new shape needs to be added can be nontrivial.

Also, consider the kind of changes that would have to be made. We'd have to add a new member to the `ShapeType enum`. Since all the different shapes depend on the declaration of this `enum`, we'd have to recompile them all.[4] And we'd also have to recompile all the modules that depend on `Shape`.

So, not only must we change the source code of all `switch/case` statements or `if/else` chains, but we also must alter the binary files (via recompilation) of all the modules that use any of the `Shape` data structures. Changing the binary files means that any DLLs, shared libraries, or other kinds of binary components must be redeployed. The simple act of adding a new shape to the application causes a cascade of subsequent changes to many source modules and to even more binary modules and binary components. Clearly, the impact of adding a new shape is very large.

Bad Design. Let's run through this again. The solution in Listing 9-1 is Rigid because the addition of `Triangle` causes `Shape`, `Square`, `Circle`, and `DrawAllShapes` to be recompiled and redeployed. It is Fragile because there will be many other `switch/case` or `if/else` statements that are both hard to find and hard to decipher. It is Immobile because anyone attempting to reuse `DrawAllShapes` in another program is required to bring along `Square` and `Circle`, even if that new program does not need them. Thus, Listing 9-1 exhibits many of the smells of bad design.



Conforming to the OCP

Listing 9-2 shows the code for a solution to the `square/circle` problem that conforms to the OCP. In this case, we have written an abstract class named `Shape`. This abstract class has a single abstract method named `Draw`. Both `Circle` and `Square` are derivatives of the `Shape` class.

Listing 9-2

OOD solution to Square/Circle problem.

```
class Shape
{
  public:
    virtual void Draw() const = 0;
};
```

4. Changes to enums can cause a change in the size of the variable used to hold the enum. So, great care must be taken if you decide that you don't really need to recompile the other shape declarations.

```
class Square : public Shape
{
  public:
    virtual void Draw() const;
};

class Circle : public Shape
{
  public:
    virtual void Draw() const;
};

void DrawAllShapes(vector<Shape*>& list)
{
  vector<Shape*>::iterator i;
  for (i=list.begin(); i != list.end(); i++)
     (*i)->Draw();
}
```

Note that if we want to extend the behavior of the `DrawAllShapes` function in Listing 9-2 to draw a new kind of shape, all we need do is add a new derivative of the `Shape` class. The `DrawAllShapes` function does not need to change. Thus `DrawAllShapes` conforms to the OCP. Its behavior can be extended without modifying it. Indeed, adding a `Triangle` class has *absolutely no effect* on any of the modules shown here. Clearly some part of the system must change in order to deal with the `Triangle` class, but all of the code shown here is immune to the change.

In a real application, the `Shape` class would have many more methods. Yet adding a new shape to the application is still quite simple since all that is required is to create the new derivative and implement all its functions. There is no need to hunt through all of the application looking for places that require changes. This solution is not Fragile.

Nor is the solution Rigid. No existing source modules need to be modified, and with one exception, no existing binary modules need to be rebuilt. The module that actually creates instances of the new derivative of `Shape` must be modified. Typically, this is either done by `main`, in some function called by `main`, or in the method of some object created by `main`.[5]

Finally, the solution is not Immobile. `DrawAllShapes` can be reused by any application without the need to bring `Square` or `Circle` along for the ride. Thus, the solution exhibits none of the attributes of bad design mentioned previously.

This program conforms to the OCP. *It is changed by adding new code rather than by changing existing code.* Therefore, it does not experience the cascade of changes exhibited by nonconforming programs. The only changes required are the addition of the new module and the change related to `main` that allows the new objects to be instantiated.

OK, I Lied

The previous example was blue sky and apple pie! Consider what would happen to the `DrawAllShapes` function from Listing 9-2 if we decided that *all* `Circles` *should be drawn before any* `Squares`. The `DrawAllShapes` function is not closed against a change like this. To implement that change, we'll have to go into `DrawAllShapes` and scan the list first for `Circles` and then again for `Squares`.

5. Such objects are known as *factories*, and we'll have more to say about them in Chapter 21 on page 269.

Anticipation and "Natural" Structure

Had we anticipated this kind of change, then we could have invented an abstraction that protected us from it. The abstractions we chose in Listing 9-2 are more of a hindrance to this kind of change than a help. You may find this surprising. After all, what could be more natural than a `Shape` base class with `Square` and `Circle` derivatives? Why isn't that natural model the best one to use? Clearly the answer is that the model is *not* natural in a system where ordering is more significant than shape type.

This leads us to a disturbing conclusion. In general, no matter how "closed" a module is, there will always be some kind of change against which it is not closed. *There is no model that is natural to all contexts!*



Since closure cannot be complete, it must be strategic. That is, the designer must choose the kinds of changes against which to close his design. He must guess at the most likely kinds of changes, and then construct abstractions to protect him from those changes.

This takes a certain amount of prescience derived from experience. The experienced designer hopes he knows the users and the industry well enough to judge the probability of different kinds of changes. He then invokes the OCP against the most probable changes.

This is not easy. It amounts to making educated guesses about the likely kinds of changes that the application will suffer over time. When the developers guess right, they win. When they guess wrong, they lose. And they will certainly guess wrong much of the time.

Also, conforming to the OCP is expensive. It takes development time and effort to create the appropriate abstractions. Those abstractions also increase the complexity of the software design. There is a limit to the amount of abstraction that the developers can afford. Clearly, we want to limit the application of the OCP to changes that are likely.

How do we know which changes are likely? We do the appropriate research, we ask the appropriate questions, and we use our experience and common sense. And after all that, *we wait until the changes happen!*

Putting the "Hooks" In

How do we protect ourselves from changes? In the previous century, we had a saying. We'd "put the hooks in" for changes that we thought might take place. We felt that this would make our software flexible.

However, the hooks we put in were often incorrect. Worse, they smelled of Needless Complexity that had to be supported and maintained, even though they weren't used. This is not a good thing. We don't want to load the design with lots of unnecessary abstraction. Rather, we often wait until we actually need the abstraction, and then we put it in.

Fool Me Once... There is an old saying: "Fool me once, shame on you. Fool me twice, shame on me." This is a powerful attitude in software design. To keep from loading our software with Needless Complexity, we may permit ourselves to be fooled *once*. This means we initially write our code expecting it not to change. When a change occurs, we implement the abstractions that protect us from future changes *of that kind*. In short, we *take the first bullet*, and then we make sure we are protected from any more bullets coming from that gun.

Stimulating Change. If we decide to take the first bullet, then it is to our advantage to get the bullets flying early and frequently. We want to know what kinds of changes are likely before we are very far down the development path. The longer we wait to find out what kinds of changes are likely, the harder it will be to create the appropriate abstractions.

Therefore, we need to stimulate the changes. We do this through several of the means we discussed in Chapter 2.

- We write tests first. Testing is one kind of usage of the system. By writing tests first we force the system to be testable. Therefore changes in testability will not surprise us later. We will have built the abstractions that make the system testable. We are likely to find that many of these abstractions will protect us from other kinds of changes later.
- We develop using very short cycles—days instead of weeks.
- We develop features before infrastructure and frequently show those features to stakeholders.
- We develop the most important features first.
- We release the software early and often. We get it in front of our customers and users as quickly and as often as possible.

Using Abstraction to Gain Explicit Closure

OK, so we've taken the first bullet. The user wants us to draw all `Circles` before any `Squares`. Now we want to protect ourselves from any future changes of that kind.

How can we close the `DrawAllShapes` function against changes in the ordering of drawing? Remember that closure is based upon abstraction. Thus, in order to close `DrawAllShapes` against ordering, we need some kind of "ordering abstraction." This abstraction would provide an abstract interface through which any possible ordering policy could be expressed.

An ordering policy implies that, given any two objects, it is possible to discover which ought to be drawn first. We can define an abstract method of `Shape` named `Precedes`. This function takes another `Shape` as an argument and returns a `bool` result. The result is `true` if the `Shape` object that receives the message should be drawn before the `Shape` object passed as the argument.

In C++, this function could be represented by an overloaded `operator<` function. Listing 9-3 shows what the `Shape` class might look like with the ordering methods in place.

Now that we have a way to determine the relative ordering of two `Shape` objects, we can sort them and then draw them in order. Listing 9-4 shows the C++ code that does this.

Listing 9-3

Shape with ordering methods

```
class Shape
{
  public:
    virtual void Draw() const = 0;
    virtual bool Precedes(const Shape&) const = 0;

    bool operator<(const Shape& s) {return Precedes(s);}
};
```

Listing 9-4

DrawAllShapes with Ordering

```
template <typename P>
class Lessp // utility for sorting containers of pointers.
{
  public:
    bool operator()(const P p, const P q) {return (*p) < (*q);}
};

void DrawAllShapes(vector<Shape*>& list)
{
    vector<Shape*> orderedList = list;
```

```
    sort(orderedList.begin(),
         orderedList.end(),
         Lessp<Shape*>());

    vector<Shape*>::const_iterator i;
    for (i=orderedList.begin(); i != orderedList.end(); i++)
        (*i)->Draw();
}
```

This gives us a means for ordering `Shape` objects and for drawing them in the appropriate order. But we still do not have a decent ordering abstraction. As it stands, the individual `Shape` objects will have to override the `Precedes` method in order to specify ordering. How would this work? What kind of code would we write in `Circle::Precedes` to ensure that `Circles` were drawn before `Squares`? Consider Listing 9–5.

Listing 9-5

`Ordering a Circle`

```
bool Circle::Precedes(const Shape& s) const
{
    if (dynamic_cast<Square*>(s))
        return true;
    else
        return false;
}
```

It should be very clear that this function, and all its siblings in the other derivatives of `Shape`, do not conform to the OCP. There is no way to close them against new derivatives of `Shape`. Every time a new derivative of `Shape` is created, all the `Precedes()` functions will need to be changed.[6]

Of course this doesn't matter if no new derivatives of `Shape` are ever created. On the other hand, if they are created frequently, this design would cause a significant amount of thrashing. Again, we'd take the first bullet.

Using a "Data-Driven" Approach to Achieve Closure

If we must close the derivatives of `Shape` from knowledge of each other, we can use a table-driven approach. Listing 9-6 shows one possibility.

Listing 9-6

`Table driven type ordering mechanism`

```
#include <typeinfo>
#include <string>
#include <iostream>

using namespace std;

class Shape
{
  public:
    virtual void Draw() const = 0;
    bool Precedes(const Shape&) const;
```

6. It is possible to solve this problem by using the ACYCLIC VISITOR pattern described in Chapter 29. Showing that solution now would be getting ahead of ourselves a bit. I'll remind you to come back here at the end of that chapter.

```
    bool operator<(const Shape& s) const
    {return Precedes(s);}
  private:
    static const char* typeOrderTable[];
};

const char* Shape::typeOrderTable[] =
{
    typeid(Circle).name(),
    typeid(Square).name(),
    0
};

// This function searches a table for the class names.
// The table defines the order in which the
// shapes are to be drawn. Shapes that are not
// found always precede shapes that are found.
//
bool Shape::Precedes(const Shape& s) const
{
    const char* thisType = typeid(*this).name();
    const char* argType = typeid(s).name();
    bool done = false;
    int thisOrd = -1;
    int argOrd = -1;
    for (int i=0; !done; i++)
    {
        const char* tableEntry = typeOrderTable[i];
        if (tableEntry != 0)
        {
            if (strcmp(tableEntry, thisType) == 0)
                thisOrd = i;
            if (strcmp(tableEntry, argType) == 0)
                argOrd = i;
            if ((argOrd >= 0) && (thisOrd >= 0))
                done = true;
        }
        else // table entry == 0
            done = true;
    }
    return thisOrd < argOrd;
}
```

By taking this approach, we have successfully closed the `DrawAllShapes` function against ordering issues in general and each of the `Shape` derivatives against the creation of new `Shape` derivatives or a change in policy that reorders the `Shape` objects by their type. (e.g., changing the ordering so that `Squares` are drawn first.)

The only item that is not closed against the order of the various `Shapes` is the table itself. That table can be placed in its own module, separate from all the other modules, so that changes to it do not affect any of the other modules. Indeed, in C++, we can choose which table to use at link time.

Conclusion

In many ways, the OCP is at the heart of object-oriented design. Conformance to this principle is what yields the greatest benefits claimed for object oriented technology (i.e., flexibility, reusability, and maintainability). Yet conformance to this principle is not achieved simply by using an object-oriented programming language. Nor is it a

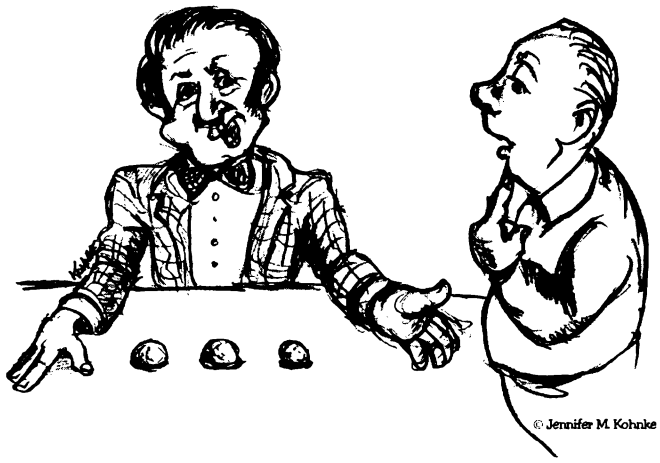
good idea to apply rampant abstraction to every part of the application. Rather, it requires a dedication on the part of the developers to apply abstraction only to those parts of the program that exhibit frequent change. *Resisting premature abstraction is as important as abstraction itself.*

Bibliography

1. Jacobson, Ivar, et al. *Object-Oriented Software Engineering*. Reading, MA: Addison–Wesley, 1992.
2. Meyer, Bertrand. *Object-Oriented Software Construction*, 2d ed. Upper Saddle River, NJ: Prentice Hall, 1997.

10

LSP: The Liskov Substitution Principle



The primary mechanisms behind the OCP are abstraction and polymorphism. In statically typed languages like C++ and Java, one of the key mechanisms that supports abstraction and polymorphism is inheritance. It is by using inheritance that we can create derived classes that implement abstract methods in base classes.

What are the design rules that govern this particular use of inheritance? What are the characteristics of the best inheritance hierarchies? What are the traps that will cause us to create hierarchies that do not conform to the OCP? These are the questions that are addressed by the Liskov Substitution Principle (LSP).

LSP: The Liskov Substitution Principle

The LSP can be paraphrased as follows:

> *SUBTYPES MUST BE SUBSTITUTABLE FOR THEIR BASE TYPES.*

Barbara Liskov first wrote this principle in 1988.[1] She said,

> *What is wanted here is something like the following substitution property: If for each object o_1 of type S there is an object o_2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o_1 is substituted for o_2 then S is a subtype of T.*

1. [Liskov88].

The importance of this principle becomes obvious when you consider the consequences of violating it. Presume that we have a function *f* that takes, as its argument, a pointer or reference to some base class *B*. Presume also that there is some derivative *D* of *B* which, when passed to *f* in the guise of *B*, causes *f* to misbehave. Then *D* violates the LSP. Clearly *D* is Fragile in the presence of *f*.

The authors of *f* will be tempted to put in some kind of test for *D* so that *f* can behave properly when a *D* is passed to it. This test violates the OCP because now *f* is not closed to all the various derivatives of *B*. Such tests are a code smell that are the result of inexperienced developers (or, what's worse, developers in a hurry) reacting to LSP violations.

A Simple Example of a Violation of the LSP

Violating the LSP often results in the use of Run-Time Type Information (RTTI) in a manner that grossly violates the OCP. Frequently, an explicit `if` statement or `if/else` chain is used to determine the type of an object so that the behavior appropriate to that type can be selected. Consider Listing 10-1.

Listing 10-1

A violation of LSP causing a violation of OCP.

```
struct Point {double x,y;};

struct Shape {
  enum ShapeType {square, circle} itsType;
  Shape(ShapeType t) : itsType(t) {}
};

struct Circle : public Shape
{
  Circle() : Shape(circle) {};
  void Draw() const;
  Point itsCenter;
  double itsRadius;
};

struct Square : public Shape
{
  Square() : Shape(square) {};
  void Draw() const;
  Point itsTopLeft;
  double itsSide;
};

void DrawShape(const Shape& s)
{
  if (s.itsType == Shape::square)
    static_cast<const Square&>(s).Draw();
  else if (s.itsType == Shape::circle)
    static_cast<const Circle&>(s).Draw();
}
```

Clearly, the `DrawShape` function in Listing 10-1 violates the OCP. It must know about every possible derivative of the `Shape` class, and it must be changed whenever new derivatives of `Shape` are created. Indeed, many rightly view the structure of this function as anathema to good design. What would drive a programmer to write a function like this?

Consider Joe the Engineer. Joe has studied object-oriented technology and has come to the conclusion that the overhead of polymorphism is too high to pay.[2] Therefore, he defined class `Shape` without any virtual functions. The classes (structs) `Square` and `Circle` derive from `Shape` and have `Draw()` functions, but they don't override a function in `Shape`. Since `Circle` and `Square` are not substitutable for `Shape`, `DrawShape` must inspect its incoming `Shape`, determine its type, and then call the appropriate `Draw` function.

The fact that `Square` and `Circle` cannot be substituted for `Shape` is a violation of the LSP. This violation forced the violation of the OCP by `DrawShape`. Thus, *a violation of LSP is a latent violation of OCP.*

Square and Rectangle, a More Subtle Violation

Of course, there are other, far more subtle, ways of violating the LSP. Consider an application which uses the `Rectangle` class as described in Listing 10-2.

Listing 10-2

`Rectangle class`

```
class Rectangle
{
  public:
    void   SetWidth(double w)  {itsWidth=w;}
    void   SetHeight(double h) {itsHeight=w;}
    double GetHeight() const   {return itsHeight;}
    double GetWidth() const    {return itsWidth;}
  private:
    Point  itsTopLeft;
    double itsWidth;
    double itsHeight;
};
```

Imagine that this application works well and is installed in many sites. As is the case with all successful software, its users demand changes from time to time. One day, the users demand the ability to manipulate *squares* in addition to rectangles.

It is often said that inheritance is the *IS-A* relationship. In other words, if a new kind of object can be said to fulfill the IS-A relationship with an old kind of object, then the class of the new object should be derived from the class of the old object.

For all normal intents and purposes, a square *is a* rectangle. Thus, it is logical to view the `Square` class as being derived from the `Rectangle` class. (See Figure 10-1.)

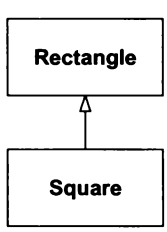


Figure 10-1 Square inherits from Rectangle

This use of the IS-A relationship is sometimes thought to be one of the fundamental techniques of object-oriented analysis:[3] A square is a rectangle, and so the `Square` class should be derived from the `Rectangle` class.

2. On a reasonably fast machine, that overhead is on the order of 1ns per method invocation, so it's hard to see Joe's point.

3. A term that is frequently used but seldom defined.

However, this kind of thinking can lead to some subtle, yet significant, problems. Generally, these problem are not foreseen until we see them in code.

Our first clue that something has gone wrong might be the fact that a `Square` does not need both `itsHeight` and `itsWidth` member variables. Yet it will inherit them from `Rectangle`. Clearly, this is wasteful. In many cases, such waste is insignificant. But if we must create hundreds of thousands of `Square` objects (e.g., a CAD/CAE program in which every pin of every component of a complex circuit is drawn as a square), this waste could be significant.

Let's assume, for the moment, that we are not very concerned with memory efficiency. There are other problems that ensue from deriving `Square` from `Rectangle`. `Square` will inherit the `SetWidth` and `SetHeight` functions. These functions are inappropriate for a `Square`, since the width and height of a square are identical. This is a strong indication that there is a problem. However, there is a way to sidestep the problem. We could override `SetWidth` and `SetHeight` as follows:

```
void Square::SetWidth(double w)
{
  Rectangle::SetWidth(w);
  Rectangle::SetHeight(w);
}

void Square::SetHeight(double h)
{
  Rectangle::SetHeight(h);
  Rectangle::SetWidth(h);
}
```

Now, when someone sets the width of a `Square` object, its height will change correspondingly. And when someone sets the height, its width will change with it. Thus, the invariants[4] of the `Square` remain intact. The `Square` object will remain a mathematically proper square.

```
Square s;
s.SetWidth(1); // Fortunately sets the height to 1 too.
s.SetHeight(2); // sets width and height to 2. Good thing.
```

But consider the following function:

```
void f(Rectangle& r)
{
  r.SetWidth(32); // calls Rectangle::SetWidth
}
```

If we pass a reference to a `Square` object into this function, the `Square` object will be corrupted because the height won't be changed. This is a clear violation of LSP. The `f` function does not work for derivatives of its arguments. The reason for the failure is that `SetWidth` and `SetHeight` were not declared `virtual` in `Rectangle`; therefore, they are not polymorphic.

We can fix this easily. However, when the creation of a derived class causes us to make changes to the base class, it often implies that the design is faulty. Certainly it violates the OCP. We might counter this by saying that forgetting to make `SetWidth` and `SetHeight` `virtual` was the real design flaw, and we are just fixing it now. However, this is hard to justify since setting the height and width of a rectangle are exceedingly primitive operations. By what reasoning would we make them `virtual` if we did not anticipate the existence of `Square`.

4. Those properties that must always be true regardless of state.

Still, let's assume that we accept the argument and fix the classes. We wind up with the code in Listing 10-3.

Listing 10-3

Rectangle and Square that are Self-Consistent.

```
class Rectangle
{
  public:
    virtual void SetWidth(double w)  {itsWidth=w;}
    virtual void SetHeight(double h) {itsHeight=h;}
    double       GetHeight() const   {return itsHeight;}
    double       GetWidth() const    {return itsWidth;}
  private:
    Point  itsTopLeft
    double itsHeight;
    double itsWidth;
};

class Square : public Rectangle
{
  public:
    virtual void SetWidth(double w);
    virtual void SetHeight(double h);
};

void Square::SetWidth(double w)
{
  Rectangle::SetWidth(w);
  Rectangle::SetHeight(w);
}

void Square::SetHeight(double h)
{
  Rectangle::SetHeight(h);
  Rectangle::SetWidth(h);
}
```

The Real Problem

`Square` and `Rectangle` now appear to work. No matter what you do to a `Square` object, it will remain consistent with a mathematical square. And regardless of what you do to a `Rectangle` object, it will remain a mathematical rectangle. Moreover, you can pass a `Square` into a function that accepts a pointer or reference to a `Rectangle`, and the `Square` will still act like a square and will remain consistent.

Thus, we might conclude that the design is now self-consistent and correct. However, this conclusion would be amiss. A design that is self-consistent is not necessarily consistent with all its users! Consider the following function `g`:

```
void g(Rectangle& r)
{
  r.SetWidth(5);
  r.SetHeight(4);
  assert(r.Area() == 20);
}
```

This function invokes the `SetWidth` and `SetHeight` members of what it believes to be a `Rectangle`. The function works just fine for a `Rectangle`, but it declares an assertion error if passed a `Square`. So here is the real problem: *The author of* `g` *assumed that changing the width of a* `Rectangle` *leaves its height unchanged.*

Clearly, it is reasonable to assume that changing the width of a rectangle does not affect its height! However, not all objects that can be passed as `Rectangle`s satisfy that assumption. If you pass an instance of a `Square` to a function like `g`, whose author made that assumption, then that function will malfunction. Function `g` is Fragile with respect to the `Square`/`Rectangle` hierarchy.

Function `g` shows that there exist functions that take pointers or references to `Rectangle` objects, but that cannot operate properly on `Square` objects. Since, for these functions, `Square` is not substitutable for `Rectangle`, the relationship between `Square` and `Rectangle` violates the LSP.

One might contend that the problem lay in function `g`—that the author had no right to make the assumption that width and height were independent. The author of `g` would disagree. The function `g` takes a `Rectagle` as its argument. There are invariants, statements of truth, that obviously apply to a class named `Rectangle`, and one of those invariants is that height and width are independent. The author of `g` had every right to assert this invariant. It is the author of `Square` that has violated the invariant.

Interestingly enough, the author of `Square` did not violate an invariant of `Square`. By deriving `Square` from `Rectangle`, the author of `Square` violated an invariant of `Rectangle`!

Validity Is Not Intrinsic



The LSP leads us to a very important conclusion: *A model, viewed in isolation, cannot be meaningfully validated.* The validity of a model can only be expressed in terms of its clients. For example, when we examined the final version of the `Square` and `Rectangle` classes in isolation, we found that they were self-consistent and valid. Yet when we looked at them from the viewpoint of a programmer who made reasonable assumptions about the base class, the model broke down.

When considering whether a particular design is appropriate or not, one cannot simply view the solution in isolation. One must view it in terms of the reasonable assumptions made by the users of that design.[5]

Who knows what reasonable assumptions the users of a design are going to make? Most such assumptions are not easy to anticipate. Indeed, if we tried to anticipate them all, we'd likely wind up imbuing our system with the smell of Needless Complexity. Therefore, like all other principles, it is often best to defer all but the most obvious LSP violations until the related Fragility has been smelled.

ISA Is about Behavior

So what happened? Why did the apparently reasonable model of the `Square` and `Rectangle` go bad? After all, isn't a `Square` a `Rectangle`? Doesn't the IS-A relationship hold?

Not as far as the author of `g` is concerned! A square might be a rectangle, but from `g`'s point of view, a `Square` object is definitely *not* a `Rectangle` object. Why? Because the *behavior* of a `Square` object is not consistent with `g`'s expectation of the behavior of a `Rectangle` object. Behaviorally, a `Square` is not a `Rectangle`, and it is *behavior* that software is really all about. The LSP makes it clear that in OOD, the IS-A relationship pertains to *behavior* that can be reasonably assumed and that clients depend on.

5. Often you will find that those reasonable assumptions are asserted in the unit tests written for the base class. Yet another good reason to practice test-driven development.

Design by Contract

Many developers may feel uncomfortable with the notion of behavior that is "reasonably assumed." How do you know what your clients will really expect? There is a technique for making those reasonable assumptions explicit, thereby enforcing the LSP. The technique is called design by contract (DBC) and is expounded by Bertrand Meyer.[6]

Using DBC, the author of a class explicitly states the contract for that class. The contract informs the author of any client code about the behaviors that can be relied on. The contract is specified by declaring preconditions and postconditions for each method. The preconditions must be true in order for the method to execute. On completion, the method guarantees that the postconditions are true.

We can view the postcondition of `Rectangle::SetWidth(double w)` as follows:

```
assert((itsWidth == w) && (itsHeight == old.itsHeight));
```

In this example, `old` is the value of the `Rectangle` before `SetWidth` is called. Now the rule for preconditions and postconditions of derivatives, as stated by Meyer, is:

> *A routine redeclaration [in a derivative] may only replace the original precondition by one equal or weaker, and the original postcondition by one equal or stronger.*[7]

In other words, when using an object through its base-class interface, the user knows only the preconditions and postconditions of the base class. Thus, derived objects must not expect such users to obey preconditions that are stronger than those required by the base class. That is, they must accept anything that the base class could accept. Also, derived classes must conform to all the postconditions of the base. That is, their behaviors and outputs must not violate any of the constraints established for the base class. Users of the base class must not be confused by the output of the derived class.

Clearly, the postcondition of `Square::SetWidth(double w)` is weaker[8] than the postcondition of `Rectangle::SetWidth(double w)`, since it does not enforce the constraint, `(itsHeight == old.itsHeight)`. Thus, `the SetWidth method of Square` violates the contract of the base class.

Certain languages, like Eiffel, have direct support for preconditions and postconditions. You can declare them and have the runtime system verify them for you. Neither C++ nor Java has such a feature. In these languages, we must manually consider the preconditions and postcondition of each method and make sure that Meyer's rule is not violated. Moreover, it can be very helpful to document these preconditions and postconditions in the comments for each method.

Specifying Contracts in Unit Tests

Contracts can also be specified by writing unit tests. By thoroughly testing the behavior of a class, the unit tests make the behavior of the class clear. Authors of client code will want to review the unit tests so that they know what to reasonably assume about the classes they are using.

A Real Example

Enough of squares and rectangles! Does the LSP have a bearing on real software? Let's look at a case study that comes from a project that I worked on a few years ago.

6. [Meyer97], Chapter 11, p. 331.
7. [Meyer97], p. 573, Assertion Redeclaration rule (1).
8. The term "weaker" can be confusing. X is weaker than Y if X does not enforce all the constraints of Y. It does not matter how many new constraints X enforces.

Motivation

In the early 1990s, I purchased a third-party class library that had some container classes. The containers were roughly related to the `Bags` and `Sets` of Smalltalk. There were two varieties of `Set` and two similar varieties of `Bag`. The first variety was called "bounded" and was based on an array. The second was called "unbounded" and was based on a linked list.

The constructor for `BoundedSet` specified the maximum number of elements the set could hold. The space for these elements was preallocated as an array within the `BoundedSet`. Thus, if the creation of the `BoundedSet` succeeded, we could be sure that it had enough memory. Since it was based on an array, it was very fast. There were no memory allocations performed during normal operation. And since the memory was preallocated, we could be sure that operating the `BoundedSet` would not exhaust the heap. On the other hand, it was wasteful of memory since it would seldom completely utilize all the space that it had preallocated.

`UnboundedSet`, on the other hand, had no declared limit on the number of elements it could hold. So long as there was heap memory avaliable, the `UnboundedSet` would continue to accept elements. Therefore, it was very flexible. It was also economical in that it only used the memory necessary to hold the elements that it currently contained. It was also slow because it had to allocate and deallocate memory as part of its normal operation. Finally, there was a danger that its normal operation could exhaust the heap.

I was unhappy with the interfaces of these third-party classes. I did not want my application code to be dependent on them because I felt that I would want to replace them with better classes later. Thus, I wrapped the third-party containers in my own abstract interface as shown in Figure 10-2.

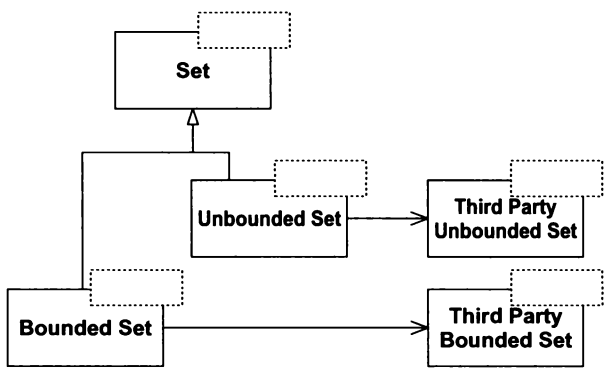


Figure 10-2 Container class adapter layer

I created an abstract class called `Set` that presented pure virtual `Add`, `Delete`, and `IsMember` functions, as shown in Listing 10-4. This structure unified the unbounded and bounded varieties of the two third-party sets and allowed them to be accessed through a common interface. Thus, some client could accept an argument of type `Set<T>&` and would not care whether the actual `Set` it worked on was of the `bounded` or `unbounded` variety. (See the `PrintSet` function in Listing 10-5.)

Listing 10-4

Abstract Set Class

```
template <class T>
class Set
{
  public:
    virtual void Add(const T&) = 0;
    virtual void Delete(const T&) = 0;
    virtual bool IsMember(const T&) const = 0;
};
```

Listing 10-5

`PrintSet`

```
template <class T>
void PrintSet(const Set<T>& s)
{
  for (Iterator<T>i(s); i; i++
    cout << (*i) << endl;
}
```

It is a big advantage not to have to know or care what kind of `Set` you are using. It means that the programmer can decide which kind of `Set` is needed in each particular instance, and none of the client functions will be affected by that decision. The programmer may choose an `UnboundedSet` when memory is tight and speed is not critical, or the programmer may choose an `BoundedSet` when memory is plentiful and speed is critical. The client functions will manipulate these objects through the interface of the base class `Set` and will therefore not know or care which kind of `Set` they are using.

Problem

I wanted to add a `PersistentSet` to this hierarchy. A persistent set is a set that can be written out to a stream and then read back in later, possibly by a different application. Unfortunately, the only third-party container that I had access to, that also offered persistence, was not a `template` class. Instead, it accepted objects that were derived from the abstract base class `PersistentObject`. I created the hierarchy shown in Figure 10-3.

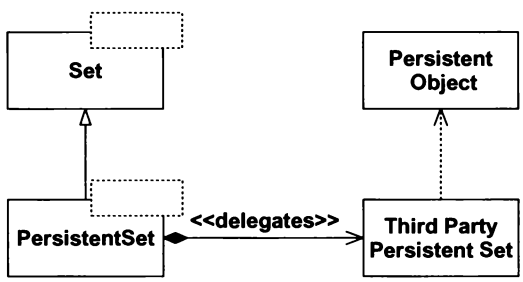


Figure 10-3 Persistent Set Hierarchy

Note that `PersistentSet` contains an instance of the third-party persistent set, to which it delegates all its methods. Thus, if you call `Add` on the `PersistentSet`, it simply delegates that to the appropriate method of the contained third-party persistent set.

On the surface of it, this might look all right. However, there is an ugly implication. Elements that are added to the third-party persistent set must be derived from `PersistentObject`. Since `PersistentSet` simply delegates to the third-party persistent set, any element added to `PersistentSet` must therefore derive from `PersistentObject`. Yet the interface of `Set` has no such constraint.

When a client is adding members to the base class `Set`, that client cannot be sure whether or not the `Set` might actually be a `PersistentSet`. Thus, the client has no way of knowing whether or not the elements it adds ought to be derived from `PersistentObject`.

Consider the code for `PersistentSet::Add()` in Listing 10-6.

Listing 10-6

```
template <typename T>
void PersistentSet::Add(const T& t)
{
  PersistentObject& p =
    dynamic_cast<PersistentObject&>(t);
  itsThirdPartyPersistentSet.Add(p);
}
```

This code makes it clear that if any client tries to add an object that is not derived from the class `PersistentObject` to my `PersistentSet`, a runtime error will ensue. The `dynamic_cast` will throw `bad_cast`. None of the existing clients of the abstract base class `Set` expects exceptions to be thrown on `Add`. Since these functions will be confused by a derivative of `Set`, this change to the hierarchy violates the LSP.

Is this a problem? Certainly. Functions that never before failed when passed a derivative of `Set` may now cause runtime errors when passed a `PersistentSet`. Debugging this kind of problem is relatively difficult since the runtime error occurs very far away from the actual logic flaw. The logic flaw is either the decision to pass a `PersistentSet` into a function or it is the decision to add an object to the `PersistentSet` that is not derived from `PersistentObject`. In either case, the actual decision might be millions of instructions away from the actual invocation of the `Add` method. Finding it can be a bear. Fixing it can be worse.

A Solution That Does *Not* Conform to the LSP

How do we solve this problem? Several years ago, I solved it by convention. Which is to say that I did not solve it in source code. Rather, I established a convention whereby `PersistentSet` and `PersistentObject` were not known to the application as a whole. They were only known to one particular module. This module was responsible for reading and writing all the containers to and from the persistent store. When a container needed to be written, its contents were copied into appropriate derivatives of `PersistentObject` and then added to `PersistentSets`, which were then saved on a stream. When a container needed to be read from a stream, the process was inverted. A `PersistentSet` was read from the stream, and then the `PersistentObjects` were removed from the `PersistentSet` and copied into regular (nonpersistent) objects, which were then added to a regular `Set`.

This solution may seem overly restrictive, but it was the only way I could think of to prevent `PersistentSet` objects from appearing at the interface of functions that would want to add nonpersistent objects to them. Moreover, it broke the dependency of the rest of the application on the whole notion of persistence.

Did this solution work? Not really. The convention was violated in several parts of the application by developers who did not understand the necessity for it. That is the problem with conventions—they have to be continually resold to each developer. If the developer has not learned the convention, or does not agree with it, then the convention will be violated. And one violation can compromise the whole structure.

An LSP-Compliant Solution

How would I solve this now? I would acknowledge that a `PersistentSet` does not have an IS-A relationship with `Set`, that it is not a proper derivative of `Set`. Thus, I would separate the hierarchies, but not completely. There are features that `Set` and `PersistentSet` have in common. In fact, it is only the `Add` method that causes the difficulty with LSP. Consequently, I would create a hierarchy in which both `Set` and `PersistentSet` were siblings beneath an abstract interface that allowed for membership testing, iteration, etc. (See Figure 10-4.) This would allow `PersistentSet` objects to be iterated and tested for membership, etc. But it would not afford the ability to add objects that were not derived from `PersistentObject` to a `PersistentSet`.

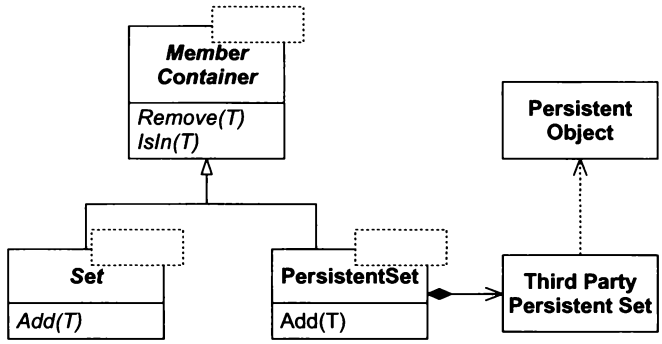


Figure 10-4 A solution that is LSP compliant

Factoring instead of Deriving

Another interesting and puzzling case of inheritance is the case of the `Line` and the `LineSegment`.[9] Consider Listings 10-7 and 10-8. These two classes appear, at first, to be natural candidates for public inheritance. `LineSegment` needs every member variable and every member function declared in `Line`. Moreover, `LineSegment` adds a new member function of its own, `GetLength`, and overrides the meaning of the `IsOn` function. Yet these two classes violate the LSP in a subtle way.

Listing 10-7

geometry/line.h

```
#ifndef GEOMETRY_LINE_H
#define GEOMETRY_LINE_H
#include "geometry/point.h"

class Line
{
  public:
    Line(const Point& p1, const Point& p2);

    double       GetSlope()         const;
    double       GetIntercept()     const; // Y Intercept
    Point        GetP1()            const {return itsP1;};
    Point        GetP2()            const {return itsP2;};
    virtual bool IsOn(const Point&) const;

  private:
    Point itsP1;
    Point itsP2;
};
#endif
```

Listing 10-8

geometry/lineseg.h

```
#ifndef GEOMETRY_LINESEGMENT_H
#define GEOMETRY_LINESEGMENT_H
class LineSegment : public Line
```

9. Despite the similarity that this example has to the Square/Rectangle example, it comes from a real application and was subject to the real problems discussed.

```
{
  public:
    LineSegment(const Point& p1, const Point& p2);
    double       GetLength()        const;
    virtual bool IsOn(const Point&) const;
};
#endif
```

A user of `Line` has a right to expect that all points that are colinear with it are on it. For example, the point returned by the `Intercept` function is the point at which the line intersects the y-axis. Since this point is collinear with the line, users of `Line` have a right to expect that `IsOn(Intercept()) == true`. In many instances of `LineSegment`, however, this statement will fail.

Why is this an important issue? Why not simply derive `LineSegment` from `Line` and live with the subtle problems? This is a judgment call. There are *rare* occasions when it is more expedient to accept a subtle flaw in polymorphic behavior than to attempt to manipulate the design into complete LSP compliance. Accepting compromise instead of pursuing perfection is an engineering trade-off. A good engineer learns when compromise is more *profitable* than perfection. However, conformance to the LSP *should not be surrendered lightly.* The guarantee that a subclass will always work where its base classes are used is a powerful way to manage complexity. Once it is forsaken, we must consider each subclass individually.

In the case of the `Line` and `LineSegment`, there is a simple solution that illustrates an important tool of OOD. If we have access to both the `Line` and `LineSegment` classes, then we can *factor* the common elements of both into an abstract base class. Listings 10-9 through 10-11 show the factoring of `Line` and `LineSegment` into the base class `LinearObject`.

Listing 10-9

geometry/linearobj.h

```
#ifndef GEOMETRY_LINEAR_OBJECT_H
#define GEOMETRY_LINEAR_OBJECT_H

#include "geometry/point.h"

class LinearObject
{
  public:
    LinearObject(const Point& p1, const Point& p2);

    double   GetSlope() const;
    double   GetIntercept() const;

    Point GetP1() const {return itsP1;};
    Point GetP2() const {return itsP2;};
    virtual int   IsOn(const Point&) const = 0; // abstract.

  private:
    Point itsP1;
    Point itsP2;
};
#endif
```

Listing 10-10

geometry/line.h

```
#ifndef GEOMETRY_LINE_H
#define GEOMETRY_LINE_H
#include "geometry/linearobj.h"

class Line : public LinearObject
{
  public:
    Line(const Point& p1, const Point& p2);
    virtual bool IsOn(const Point&) const;
};
#endif
```

Listing 10-11

geometry/lineseg.h

```
#ifndef GEOMETRY_LINESEGMENT_H
#define GEOMETRY_LINESEGMENT_H
#include "geometry/linearobj.h"

class LineSegment : public LinearObject
{
  public:
    LineSegment(const Point& p1, const Point& p2);

    double       GetLength()        const;
    virtual bool IsOn(const Point&) const;
};
#endif
```

`LinearObject` represents both `Line` and `LineSegment`. It provides most of the functionality and data members for both subclasses, with the exception of the `IsOn` method, which is pure virtual. Users of `LinearObject` are not allowed to assume that they understand the extent of the object they are using. Thus, they can accept either a `Line` or a `LineSegment` with no problem. Moreover, users of `Line` will never have to deal with a `LineSegment`.

Factoring is a design tool that is best applied before there is much code written. Certainly, if there were dozens of clients of the `Line` class shown in Listing 10-7, we would not have had an easy time of factoring out the `LinearObject` class. When factoring is possible, however, it is a powerful tool. If qualities can be factored out of two subclasses, there is the distinct possibility that other classes will show up later that need those qualities, too. Of factoring, Rebecca Wirfs–Brock, Brian Wilkerson, and Lauren Wiener say:

> *We can state that if a set of classes all support a common responsibility, they should inherit that responsibility from a common superclass.*
> *If a common superclass does not already exist, create one, and move the common responsibilities to it. After all, such a class is demonstrably useful—you have already shown that the responsibilities will be inherited by some classes. Isn't it conceivable that a later extension of your system might add a new subclass that will support those same responsibilities in a new way? This new superclass will probably be an abstract class.*[10]

10. [WirfsBrock90], p. 113.

Listing 10-12 shows how the attributes of `LinearObject` can be used by an unanticipated class, `Ray`. A `Ray` is substitutable for a `LinearObject`, and no user of `LinearObject` would have any trouble dealing with it.

Listing 10-12

geometry/ray.h

```
#ifndef GEOMETRY_RAY_H
#define GEOMETRY_RAY_H

class Ray : public LinearObject
{
  public:
    Ray(const Point& p1, const Point& p2);
    virtual bool IsOn(const Point&) const;
};
#endif
```

Heuristics and Conventions

There are some simple heuristics that can give you some clues about LSP violations. They all have to do with derivative classes that somehow *remove* functionality from their base classes. A derivative that does less than its base is usually not substitutable for that base, and therefore violates the LSP.

Degenerate Functions in Derivatives

Consider Listing 10-13. The `f` function in `Base` is implemented. However, in `Derived` it is degenerate. Presumably, the author of `Derived` found that function `f` had no useful purpose in a `Derived`. Unfortunately, the users of `Base` don't know that they shouldn't call `f`, so there is a substitution violation.

Listing 10-13

A degenerate function in a derivative

```
public class Base
{
  public void f() {/*some code*/}
}

public class Derived extends Base
{
  public void f() {}
}
```

The presence of degenerate functions in derivatives is not always indicative of an LSP violation, but it's worth looking at them when they occur.

Throwing Exceptions from Derivatives

Another form of violation is the addition of exceptions to methods of derived classes whose bases don't throw them. If the users of the base classes don't expect exceptions, then adding them to the methods of derivatives is not substitutable. Either the expectations of the users must be altered or the derived classes should not throw the exceptions.

Conclusion

The OCP is at the heart of many of the claims made for OOD. When this principle is in effect, applications are more maintainable, reusable, and robust. The LSP is one of the prime enablers of the OCP. It is the substitutability of subtypes that allows a module, expressed in terms of a base type, to be extensible without modification. That substitutability must be something that developers can depend on implicitly. Thus, the contract of the base type has to be well and prominently understood, if not explicitly enforced, by the code.

The term "IS-A" is too broad to act as a definition of a subtype. The true definition of a subtype is "substitutable," where substitutability is defined by either an explicit or implicit contract.

Bibliography

1. Meyer, Bertrand. *Object-Oriented Software Construction*, 2d ed. Upper Saddle River, NJ: Prentice Hall, 1997.
2. Wirfs–Brock, Rebecca, et al. *Designing Object-Oriented Software*. Englewood Cliffs, NJ: Prentice Hall, 1990.
3. Liskov, Barbara. Data Abstraction and Hierarchy. *SIGPLAN Notices*, 23,5 (May 1988).